
Title: Full Explanation of the To-Do App Code

Characters:

- **Jumar (Frontend Developer)** – Covers HTML, CSS, and JavaScript.
 - **Eric (Backend Developer)** – Explains PHP and database operations.
 - **Shintaine (Project Lead)** – Connects everything and ensures clarity.
-

Scene: A Technical Breakdown of the To-Do App

(The team is in a meeting, going through the app's files.)

Act 1: Understanding the Frontend

Jumar (HTML Structure – index.html)

"Let's start with index.html, which lays out the structure of our app."

```
html
Copy Edit

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Todo App</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>
  <h1>To-Do List</h1>
  <div>
    <input type="text" id="todo-input" placeholder="Enter a task">
    <button id="add-btn">Add</button>
  </div>
  <p id="no-tasks-message" class="no-tasks">No tasks available.</p>
  <ul class="todo-list" id="todo-list"></ul>
  |
  <h2>Completed Tasks</h2>
  <ul class="completed-list" id="completed-list"></ul>

  <script src="script.js"></script>
</body>

</html>
```

Key Components:

- The input field (#todo-input) captures user tasks.
 - The add button (#add-btn) triggers task creation.
 - #todo-list stores active tasks.
 - #completed-list displays finished tasks.
 - The page is linked to style.css for styling and script.js for interactivity.
-

Jumar (CSS Styling – style.css)

"Next, style.css makes the app visually appealing and user-friendly."

```
CSS Copy Edit

.task-item {
  background: white;
  padding: 12px;
  border-radius: 5px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 10px;
  box-shadow: 0px 2px 5px rgba(0, 0, 0, 0.1);
  transition: 0.3s;
  min-height: 50px;
  width: 350px;
}
```

- Tasks are displayed as **cards** with a soft shadow effect.
 - Buttons are styled for **smooth hover effects** and clear distinction.
-

Jumar (JavaScript – script.js)

"Now, let's move to script.js, which makes our app dynamic."

Fetching and Displaying Tasks

```
js Copy Edit  
  
const API_URL = 'http://localhost:3000/api/todos.php';  
  
async function fetchTodos() {  
  const response = await fetch(API_URL);  
  const todos = await response.json();  
  displayTodos(todos);  
}
```

- Retrieves tasks from todos.php and updates the UI.

Adding a Task

```
js Copy Edit  
  
addBtn.addEventListener('click', async () => {  
  const title = todoInput.value.trim();  
  if (!title) return;  
  
  const newTodo = { title: title };  
  await fetch(API_URL, {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify(newTodo)  
  });  
  
  fetchTodos();  
  todoInput.value = '';  
});
```



- Sends a **POST** request to add tasks.
 - Calls fetchTodos() to refresh the list.
-

Act 2: Backend & Database

Eric (Backend API – todos.php)

"Now, let's move to todos.php, which serves as the API for our app."

Fetching Tasks from the Database

```
php                                                                    Copy Edit

$conn = new mysqli("localhost", "root", "", "todo_app");
$result = $conn->query("SELECT * FROM tasks");
$todos = [];

while ($row = $result->fetch_assoc()) {
    $todos[] = $row;
}

echo json_encode($todos);
```

- Establishes a **MySQL connection**.
- Retrieves tasks and **returns them in JSON format**.

Adding a Task

```
php                                                                    Copy Edit

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $data = json_decode(file_get_contents("php://input"), true);
    $stmt = $conn->prepare("INSERT INTO tasks (title, completed) VALUES (?, 0)");
    $stmt->bind_param("s", $data['title']);
    $stmt->execute();
}
```

- Reads **JSON data** sent by JavaScript.
 - Inserts a **new task** into the database.
-

Shintaine (Database – todo_app.sql)

"Now, let's understand the database setup in todo_app.sql."

```
sql
CREATE TABLE tasks (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  completed BOOLEAN DEFAULT 0
);
```

- Stores tasks with:
 - **id** (unique identifier).
 - **title** (task description).
 - **completed** (boolean to track task status).

Shintaine (Completing and Deleting Tasks – todos.php)

"When a task is marked complete or deleted, todos.php handles it."

Marking a Task as Complete

```
php
if ($_SERVER["REQUEST_METHOD"] === "PUT") {
  $data = json_decode(file_get_contents("php://input"), true);
  $stmt = $conn->prepare("UPDATE tasks SET completed=? WHERE id=?");
  $stmt->bind_param("ii", $data['completed'], $data['id']);
  $stmt->execute();
}
```

- Updates task status in the database.

Deleting a Task

```
php                                                                    Copy Edit

if ($_SERVER["REQUEST_METHOD"] === "DELETE") {
    $data = json_decode(file_get_contents("php://input"), true);
    $stmt = $conn->prepare("DELETE FROM tasks WHERE id=?");
    $stmt->bind_param("i", $data['id']);
    $stmt->execute();
}
```

- Permanently **removes a task**.

Act 3: Summary & Future Improvements

Shintaine (Bringing It All Together)

"Now that we've covered every file, let's summarize how everything works."

1. **Frontend (index.html, style.css, script.js)**
 - Provides the UI.
 - Handles user interactions (adding, completing, deleting tasks).
2. **Backend (todos.php)**
 - Acts as an API to communicate with the database.
 - Handles **task retrieval, addition, updates, and deletions**.
3. **Database (todo_app.sql)**
 - Stores all tasks persistently.

Shintaine (Future Improvements)

"To make this app more powerful, we could add:"

1. **User Authentication** – Separate task lists for different users.
2. **Task Deadlines** – Assign due dates.
3. **Task Categories** – Organize tasks by priority.
4. **Drag-and-Drop Sorting** – Reorder tasks manually.

"With these improvements, we can make this To-Do App an even more powerful productivity tool!"

Final Thoughts

- **Jumar:** *"Our frontend ensures a smooth and interactive experience."*
 - **Eric:** *"The backend efficiently manages tasks and data storage."*
 - **Shintaine :** *"With everything connected, our app functions seamlessly, and there's plenty of room for enhancements!"*
-

This script **includes all the code** and ensures that **Taylor contributes to the backend explanation**. Let me know if you need any refinements! 😊